Twacha: handoff brief for the coding agent

Read this whole file first. It says what exists, what to build, what you must not change, and how to know you are done.

1. Project in one paragraph

Twacha is a skin and diet wellness tracker, a class project built by two students (about 5 to 10 hours a week each). The user takes a face photo; a backend estimates acne severity and rough skin measures, looks up local air quality, and suggests Indian foods to try with an honest evidence note. A 30-day program view shows the trend. **A working demo for the professor is due about three weeks after 12 Oct 2026**, so favour a small, reliable, honest app over extra features.

2. Current state

Part	State
Backend (FastAPI, `twacha_backend_v3.zip`)	Working and tested (14 tests). Needs deploying and small additions (section 6, task B1)
Frontend (React 18 + Vite + Capacitor 6, in the team's `web/` folder; live at twacha-skincare-routine.netlify.app)	Polished UI, **all data hard-coded, not connected to any backend**
Android build	Capacitor project exists in `web/android/`
Model	`ml/twacha_acne_model.onnx` (MobileNetV2 trained on ACNE04), loaded by the backend

You have the full frontend zip; I (the previous assistant) have not seen its source. **Start with task T0.**

3. Non-negotiable rules

1. **Honesty over polish.** This is a prototype, not a medical product. Remove or reword: "Clinical Dermatology", "clinical" anywhere, "calibrated for Fitzpatrick skin phototypes III through VI", "clinical diagnostic", "diagnosis". The model has not been validated on any skin type. Put this line on every result screen and in the footer: *"Experimental estimate for a class project. Not a medical diagnosis."* The API also returns it as `prototype_notice`.
2. **Never invent data in the app.** If the API fails, show an error. Do not fall back to fake scores or fake air quality.

3. **Photos are never stored on the server.** Do not add image storage to the backend. Any photo kept for the timeline stays on the user's device only (see T6).
4. **Air quality:** show the attribution "Air quality data: World Air Quality Index Project (aqicn.org)" wherever AQI is displayed. Hide the AQI card when `environment.source == "unavailable"`. Show a visible "demo" tag when `source == "demo"`.
5. **Do not change `backend/app/recommender.py` weights or `backend/app/evidence.py` text** without asking the team. The weights are placeholders chosen by the team; the evidence notes were checked against sources and none is labelled "clinical". Do not upgrade an `evidence_tier` to "clinical".
6. **Do not add new health claims** (for example "reduces acne by X%", "treats", "cures").
7. Ask the team before adding paid services, new third-party APIs, or analytics/trackers.
8. Keep changes small and reviewable; one task per commit.

4. API contract (backend v3)

Base URL comes from `VITE_API_BASE_URL` (for example `https://<service>.onrender.com`). All paths below are relative to it.

POST `/api/scan/`

`multipart/form-data`: `image` (required, JPEG/PNG/WebP, max 10 MB), `city` (optional string).

Response `201`:

```json
```

```
{
  "scan": {
    "id": 15, "created_at": "2026-10-12T08:06:40.647758Z", "day_number": 15,
    "acne_score": 7.6, "redness_score": 9.8, "dryness_score": 8.4, "ita_value":
  },
  "environment": {
    "local_aqi": 215, "pm25": 107.5, "dominant_pollutant": "pm25",
    "aqi_band": "poor", "source": "demo", "city": "delhi"
  },
  "nutrient_scores": { "antioxidant_vit_e": 8.61, "omega3": 8.128, "low_glycemi
  "recommendations": [
    {
      "nutrient_key": "antioxidant_vit_e", "relevance_score": 8.61,
      "title": "Add vitamin E-rich foods",
      "body": "...", "nutrient_explanation": "...",
      "evidence_tier": "mixed", "source_note": "...",
      "foods_to_add": [
        { "name": "Almonds (badam)", "reason": "...", "nutrient_value": null, "
          "gi": null, "source": null, "logged_recently": false }
      ],
      "foods_to_reduce": [],
      "trend": "worsening"
    }
  ],
  "quality_notes": ["Facial hair detected: the acne score uses the forehead and
  "prototype_notice": "Experimental estimate for a class project. Not a medical
}
```


Field meaning:

- Scores are 0 to 10, **lower is healthier.** `acne_score` can reach at most 9.5. `ita_value` is stored context only; **never show it as a score and never use it for advice.**
- `redness_score`, `dryness_score` are rough colour estimates, **not calibrated.** Label them "experimental" in the UI.
- `environment.source`: `live | demo | unavailable`. `aqi_band`: `good | satisfactory | moderate | poor | very_poor | severe`.
- `evidence_tier`: `clinical | mixed | traditional` (v3 returns `mixed` only). `trend`: `improving | flat | worsening | insufficient_data` (needs at least 3 earlier scans).
- `recommendations` has 1 to 3 items; `nutrient_scores` has all ten keys: `low_glycemic, dairy_reduce, zinc, omega3, antioxidant_vit_c, antioxidant_vit_e, healthy_fats, probiotic, vitamin_d, water_intake`.

- `quality_notes` are user-facing strings; always display them.

Errors: `422 {"detail": "<readable message>"}` (no face, too dark, blurry, not an image, no file); `413` file over 10 MB; `500` generic. Show `detail` to the user with a Retake button.

GET `/api/history/?limit=30` (limit 1 to 200)

```json
{ "program": { "start_date": "2026-09-28", "current_day": 15, "total_days": 30
  "scans": [ { "id": 15, "created_at": "...Z", "day_number": 15, "acne_score":
               "redness_score": 9.8, "dryness_score": 8.4, "ita_value": 3.0, "l
```

Newest first. `local_aqi` can be `null`. Timestamps are UTC with `Z`; convert to local time in the app.

POST `/api/log/` and GET `/api/log/?limit=7`

Body: `{"foods_eaten": ["white rice", "curd"], "log_date": "2026-10-12"}` (`log_date` optional, not in the future, at least one food). Returns `{id, log_date, foods_eaten}`. Foods logged in the last 3 days are not suggested as new additions.

GET `/api/foods/`

Sorted list of canonical food names for the log picker.

GET `/health`

`{"status":"ok","acne_model_loaded":true}`. Use it to wake the server (Render free instances sleep).

Demo only (when `ENABLE_DEMO=1`)

`POST /demo/seed?days=14` creates synthetic daily scans; `POST /demo/reset` clears all data. Any UI that shows seeded history must show a "Demo data" banner.

5. Frontend tasks (in order, each with acceptance criteria)

T0. Inventory (first, report back before coding). Read the zip. Produce `docs/SCREEN_MAP.md`: every screen/component, its hard-coded data, and which endpoint and field replaces it. Flag anything the API cannot supply (see the gaps below). *Done when* the team has reviewed it.

T1. API layer. One module (for example `src/api/client.js`) with `VITE_API_BASE_URL`, a 60-second timeout on `/api/scan/` (Render cold start), one automatic wake-up call to `/health` plus retry, and consistent error objects. *Done when* all calls go through it and a wrong URL shows a clear error.

T2. Device identity. Generate a UUID on first launch, keep it in `localStorage`, send it as `X-Device-Id` on every request. (Backend part is B1.)

T3. Scanner screen. Capture with `getUserMedia` on web and the Capacitor Camera plugin on Android. Downscale to about 1280 px on the long side, encode JPEG (this also strips EXIF), upload as multipart field `image` plus `city`. Show a tip first: daylight, face centred, no filters. Keep the mesh overlay only as a framing guide and remove any text implying it measures anything. Show a loading state and handle 422, 413, offline. *Done when* a real photo returns a result and each error shows a readable message with a Retake button.

T4. Result screen. Bind the response: three scores labelled "experimental" for redness and dryness, `quality_notes`, recommendations with evidence-tier badge and `source_note`, foods to add and reduce (badge foods with `logged_recently`), AQI card rules from section 3, disclaimer. *Done when* every field in the example above is displayed or deliberately omitted.

T5. Diet and skin plan tab. "Diet Foods" shows `foods_to_add` from the latest scan; "Skincare Products" stays static but is titled "General suggestions". Empty state before the first scan.

T6. Timeline tracker. Fill entries from `GET /api/history/`. The existing design shows photo thumbnails, **but the server does not store photos**. Choose one with the team: (a) store a small thumbnail on the device only (IndexedDB or Capacitor Filesystem), keyed by scan id, with a "stored only on this phone" note and a delete button; or (b) drop thumbnails. Default to (a).

T7. Food log screen (new). Load names from `/api/foods/`, multi-select, `POST /api/log/`. Show the last 7 days from `GET /api/log/`.

T8. Profile and greeting. Remove the hard-coded name ("Richa"): ask for a first name on first launch and store it locally. The backend computes no "barrier integrity" or "hydration" score; use the real values (acne, redness, dryness, labelled as shown in T4) or label anything else as static sample content.

T9. Copy and metadata. Change the HTML title and meta description (for example "Twacha: Skin and Dietary Wellness Tracker (prototype)"), remove clinical and Fitzpatrick wording across all screens, add the disclaimer, and describe the remedy catalogue as "traditional or home remedies, not clinically tested".

T10. Progress. Line chart of `acne_score` over time with "Day X of 30". If fewer than 3 scans, show a message instead of a chart.

T11. Android. Add camera permission to `AndroidManifest.xml`, use HTTPS only, set `capacitor.config.json` server settings for production, build a debug APK, and test on a real phone over mobile data.

T12. Deploy frontend on Netlify with `VITE_API_BASE_URL`; keep the backend `CORS_ORIGINS` in sync.

6. Backend tasks

The baseline is `twacha_backend_v3.zip` (place it at `backend/`). Run its tests first: `pip install -r requirements-dev.txt && pytest -q`.

B1. Per-device data. Add a nullable `device_id` column to `skin_scans`, `dietary_logs` and `programs`; read the `X-Device-Id` header (default `"anonymous"` if missing); filter every query by it. `create_all` does not alter existing tables, so either add a small migration or document deleting the SQLite file. Add tests proving two device IDs never see each other's history or logs.

B2. Deploy to Render. Start command `uvicorn app.main:app --host 0.0.0.0 --port $PORT`. Set `WAQI_TOKEN`, `DEFAULT_CITY`, `CORS_ORIGINS` (the Netlify URL and the Capacitor origins), `ENABLE_DEMO=1` (demo only). Free Render disks reset on redeploy; for persistence set `DATABASE_URL` to a hosted Postgres (check the provider's current free-tier terms). Add `psycopg2-binary` if Postgres is used.

B3. Keep-alive. The app calls `/health` on launch to wake the instance; do not add paid pingers.

Do **not** modify scoring, the model, or the food/evidence tables. Report problems instead.

7. Known weaknesses (do not hide them in the UI; do not try to fix silently)

- Acne model: macro-F1 0.60 and 61% accuracy on 56 test images (about 0.07 noise). Trained on clinic photos, mostly East Asian. Frontal selfies, darker skin, beards and moles can score wrongly. Beards are handled by a heuristic that scores forehead and cheeks only.
- Redness, dryness and skin tone are uncalibrated colour measures.
- Nutrient weights are team-chosen placeholders. The diet and acne evidence is mixed (a 2025 meta-analysis found no significant pooled dairy link).
- Lighting changes scores. Cold starts make the first scan slow.
- The team's own backend had extra fields (their own redness/dryness code, `metric_trends`, `escalation_notes`). Treat v3 as the baseline unless they provide theirs; ask before deleting anything.

8. Out of scope until after the demo

Accounts and passwords, retraining the model, Fitzpatrick-balanced datasets, push notifications, payments, iOS build, any claim of clinical accuracy.

9. Definition of done

- [ ] Fresh install: scan a real photo on a phone over mobile data and see the result from the deployed backend.
- [ ] No hard-coded health data remains except text labelled as general/sample content.
- [ ] Bad photo (dark, no face) shows a readable message and offers Retake.

- [ ] Food log changes the next recommendation (a logged food is no longer suggested as new).
- [ ] Timeline and Progress screens show real history; demo history shows a "Demo data" banner.
- [ ] Disclaimer and WAQI attribution visible where required; no "clinical" or "Fitzpatrick calibrated" wording anywhere.
- [ ] Backend tests pass, including the two-device isolation test; `npm run build` passes.
- [ ] `README.md` updated, `docs/SCREEN_MAP.md` present, short manual test checklist added.

10. Questions to ask the team before coding

1. Which backend files differ from v3, and should any of them be kept?
2. Thumbnails on the timeline: stored on the device only, or removed?
3. Android only, or also a web link for the demo?
4. Exact demo date (sets the feature-freeze day, one week before).
5. Do they already have a WAQI token and a Render service?